

The stuff you save to get better at your job, and never look at again.

How I designed and built an AI-powered personal knowledge system. The first version focuses on Product Management learners as its MVP audience; the architecture is built to support any learning domain.

Everyone saves things they intend to learn from, and almost nobody goes back to them. PM Content Vault is the MVP implementation of a broader knowledge system. It takes the posts, PDFs, screenshots and chat exports scattered across five apps, works out what each one is, and lets you ask questions of the whole pile, with every answer citing the saves it came from.

Role: Product & build, end to end · **Status:** Live

LIVE APP pm-content-vault.vercel.app · **SOURCE** github.com/RADHIKA2909/pm-content-vault
Read this online at pm-content-vault.vercel.app/prd-case-study

Try it without signing up: `demo@pmvault.app` / `PMVaultDemo2026!`

A demo account with one saved item of every supported type.

The problem

Anyone learning something on their own ends up with a trail of saved material. A framework posted on LinkedIn, a PDF shared in a group chat, a screenshot of a diagram, a long article bookmarked at midnight. All of it saved with real intent, and almost none of it read twice.

The obvious diagnosis is that people are disorganised and need a better folder structure. I don't think that's right. Saving isn't the part that breaks; people do that diligently enough. What breaks comes afterwards, and there are three of them.

It's scattered

LinkedIn saves, WhatsApp media, browser bookmarks, a Notes app. No single place to look, so looking anywhere feels pointless.

You can't judge it

A saved link is a title and a thumbnail. Whether it's worth ten minutes takes ten minutes to find out.

You can't find it

What you remember is *"that post about handling stakeholder conflict"*, which is never a keyword that appears in it.

THE INSIGHT THIS PRODUCT IS BUILT ON

Storage was never the bottleneck. **Retrieval and judgment** were, which means a folder app solves a problem I don't have. Everything else follows from that: it's why summaries get generated and stored the moment something is saved rather than on demand, and why semantic search got built before any organisation features.

Who it's for

Ultimately anyone who saves material to learn from later: people studying for exams, switching careers, ramping into a new role, or teaching themselves a craft. The behaviour is the same everywhere. Save with intent, never return.

The first user segment is one large, well-defined slice of that group, **Product Management learners**. The problem isn't unique to them. What they have is a shared vocabulary, a shared deadline and fairly predictable content types, and those three things are what make the classifier tractable, the taxonomy meaningful and the users reachable for research. It's the initial learning domain rather than the ceiling.

WHY THE MVP STARTS NARROW WHEN THE PROBLEM IS BROAD

A fixed taxonomy is what makes saved content instantly scannable, and a taxonomy only works if you know the domain. Build for everyone on day one and you end up with either no categories or generic ones. Both hand the job of organising back to the user, which is the thing that already failed. The good news is that **generalising later is a configuration problem rather than a rebuild**. Ingestion, embedding, retrieval and citation are already domain-agnostic. The category list is the only part that isn't.

Why build it, when so much of this exists

Plenty of tools hold saved content. None of them were built for the specific failure I had. It's worth being precise about why, rather than waving vaguely at "existing tools don't do it".

WHAT I COULD HAVE USED

WHAT IT'S BUILT FOR, AND WHERE THAT LEAVES ME

A notes app	Excellent storage, and organising is the user's job. That's precisely the step that gets skipped at the moment of saving, which is when you're mid-scroll and least willing to file anything. Any tool that asks me to organise recreates the problem.
A read-later app	Built around articles and a reading queue. Most of what I save isn't an article. It's a screenshot, a PDF, a chat message, a post. And the question it answers is "what have I not read yet", when mine is "where's that thing about stakeholder conflict".
A highlights tool	Strong at capturing passages out of things you read. It still assumes you know which source to go back to; the search is over sources, and my problem starts before I know which source it is.
A general AI chatbot	Can absolutely answer questions about a document you hand it. The gap is persistence. Every session starts empty, so the library never accumulates, nothing gets deduplicated, and nothing carries state like "already read" or "starred". A conversation isn't a corpus.
Platform search	Saved posts stay on whichever platform hosted them, so there are five searches instead of one. Each is keyword-only, and none of them know about the others.

THE GAP THIS FILLS

Each option above is strong at one of three things: *accept anything, understand it without being asked, answer across all of it with citations*. The value is in doing all three to the same pile, and they depend on each other. Understanding has to happen at save time, or retrieval has nothing to work with later.

The journey, and where it still hurts

Four moments matter. For each one, what the product does, and what friction is still there.

1 Encounter: something worth keeping, mid-scroll

On a phone, inside LinkedIn or a group chat. The window for saving is a few seconds wide. Anything more elaborate than that and the thing is simply lost.

Still friction: copy → switch app → paste

This is the one step the MVP doesn't fix, which is why share-sheet ingestion sits first on the roadmap.

2

Save: hand it over and stop thinking

Paste, upload, or drop in a chat export. Extraction, classification, summarisation, embedding and a duplicate check all run before anything is written. The result comes back for approval with every field editable.

No filing, no folders, no naming

3

Return: days later, half-remembering

Two ways in. Browse the Library, where every card carries a gist, a type and a read time, so you can judge relevance without opening anything. Or ask a question in plain language and let retrieval do the finding.

Semantic search over keyword recall

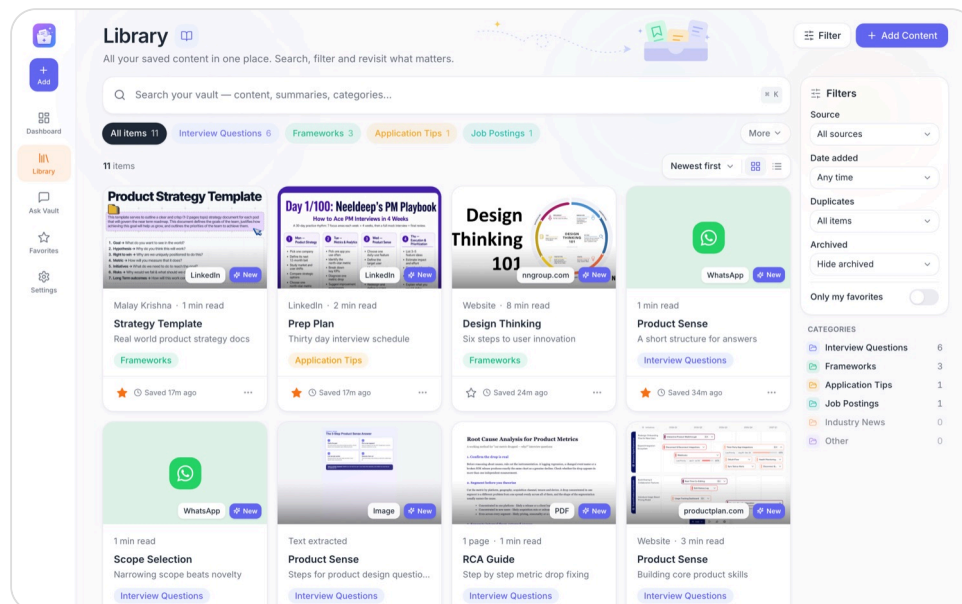
4

Verify: decide whether to trust the answer

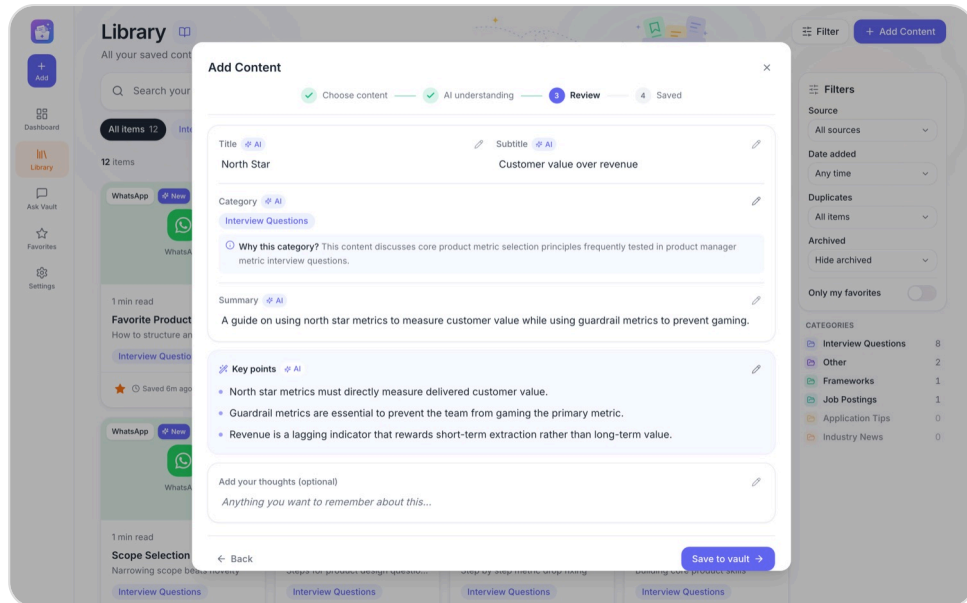
Every answer carries numbered citations. Expand one and you get the exact passage it drew on. Open it and you land on the original save, where it can be read, highlighted and annotated.

Every claim traceable to a source

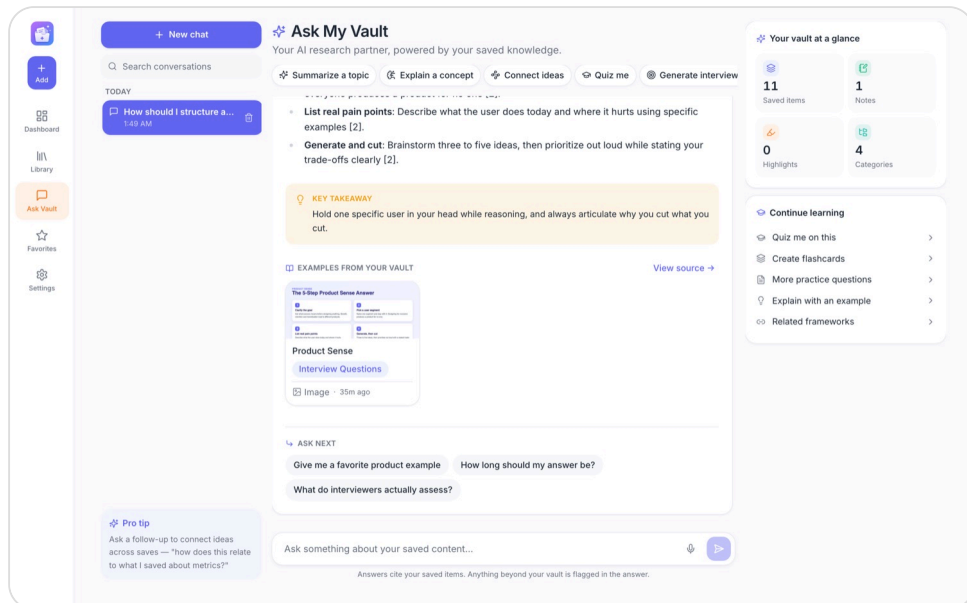
What it does



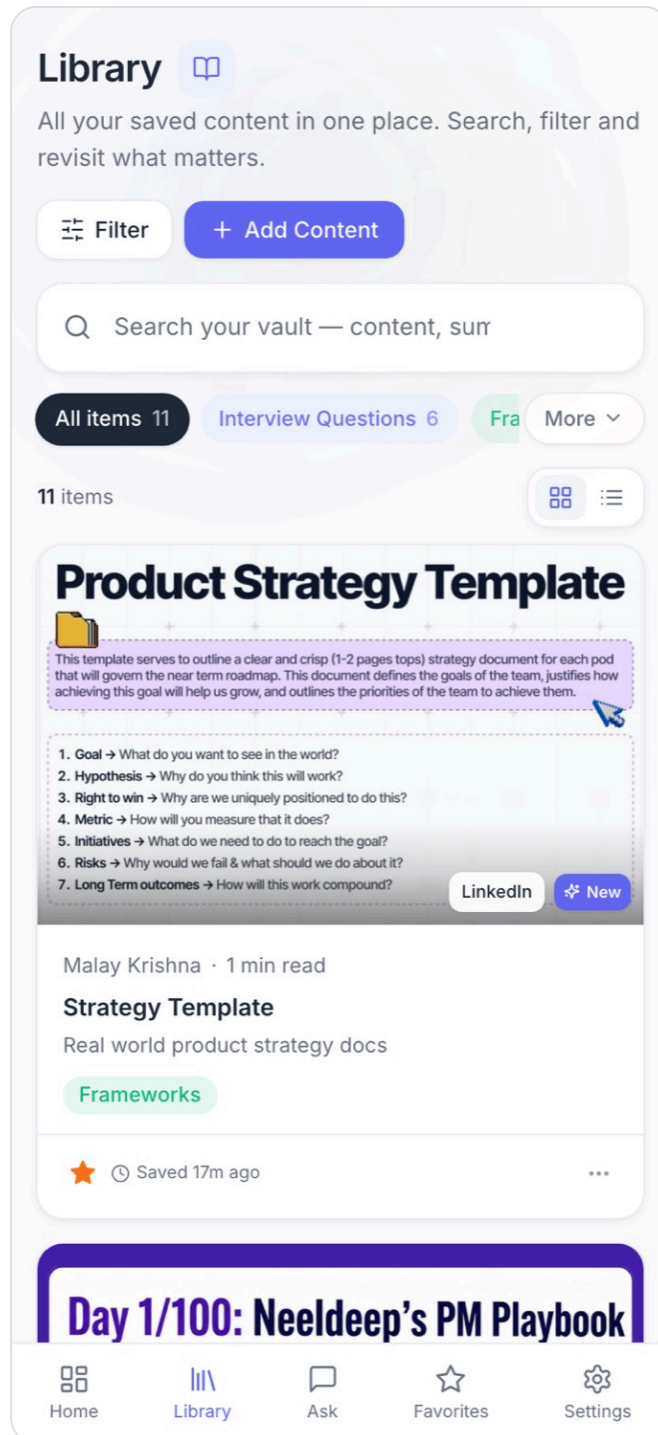
Every card knows what it is. A PDF shows its rendered first page, a screenshot shows the image, a LinkedIn post shows the author and its preview, a chat import shows the WhatsApp mark. None of it is a generic file icon, because you should be able to recognise a save without reading it. Numbers like page count and read time come from stored values, and a card leaves out any it doesn't have rather than printing a zero.



The AI proposes, you approve. Adding something runs extraction, classification, summarisation, embedding and a duplicate check, then stops and shows you what it came up with. Every field is editable. An **AI** chip marks what the model wrote, and it disappears the moment you change that field.



Answers you can check. Ask My Vault retrieves from your own saves and answers in structured sections, with inline markers pointing at numbered sources. Expand a source and you get the exact passage the answer drew on. Not the item's summary, which is a different sentence written for a different purpose.



Built for where saving actually happens. The sidebar collapses to a bottom bar and cards reflow to a single column.

Information architecture

Five destinations, plus one decision underneath them that everything else follows from.

Dashboard	: counts, recent saves, continue a conversation
Library	: everything, filtered by category / source / date / duplicates
↳ Item	: the content, its AI summary, highlights, related items
Ask My Vault	: conversations, each answer cited back into the Library
Favorites	: the starred subset, with study actions over it
Settings	: account, defaults, data

The item is the atom

A WhatsApp export doesn't become one saved item. It becomes one item *per message*. Otherwise a 40-message chat containing three useful ideas is a single unsearchable blob, and retrieval hands you the whole conversation when what you wanted was one paragraph of it. Splitting at ingest is what lets a citation point at something small enough to actually check.

Two kinds of label, on purpose

	CATEGORIES	TAGS
Who sets them	The AI, from a fixed list; the user can override	The user, freely
How many	One to three per item, never zero	Any
Why	A closed list is what makes the filter rail trustworthy. An open one gives you "Product Sense", "Product-Sense" and "PS" within a week	Personal structure the taxonomy can't anticipate. Favourites are implemented as one

NEVER ZERO

Every item lands in at least one category, falling back to `Other` if the model won't commit. An uncategorised bucket sounds harmless enough, but it's exactly where things go to be forgotten, and being forgotten is the whole problem this product exists to fix.

Decisions that shaped it

The feature list is the least interesting part of this project. What got argued down is where the thinking actually shows.

DECISION 01

RAG rather than fine-tuning or keyword search

There were three ways to do "find the right thing fast". Keyword search falls over on the exact query people actually have, because what you remember is a post's *idea* and not its vocabulary. Fine-tuning has a different problem: on a personal library that changes every day, making one new save findable means retraining. The cost is simply wrong for the use case.

Retrieval-augmented generation fits the shape of the problem. A new save is queryable the moment it's embedded, and answers stay grounded in the user's own material with citations attached. **What matters here is trustworthy recall of your own content, not general knowledge.** That framing ended up being the tiebreaker for later decisions as well. Anything that traded citation accuracy for convenience got rejected on those grounds.

DECISION 02

Nothing is written until you confirm

The natural way to build ingestion is to save the row first and enrich it in the background. I built the opposite. `/analyze` runs every AI operation against an in-memory buffer and persists nothing at all, and `/commit` is the only endpoint that writes.

Two reasons for that. An abandoned add should leave nothing behind, since half the point of this product is not accumulating junk in the first place. And a duplicate check is only useful *before* you save. Telling someone they have just created a duplicate is a notification rather than a feature.

DECISION 03

No LinkedIn scraping, even though it's the biggest source

Most of what I save is on LinkedIn, and automating that import would be the single most useful feature. Scraping your own saved posts violates LinkedIn's terms of service, so the app takes a pasted URL or pasted text and nothing else.

This costs real convenience, and I knew it would. Writing it into the brief as a locked constraint before building was the point: it meant I couldn't quietly relitigate it at 1am when the manual flow started feeling tedious.

DECISION 04

Cutting a feature I'd already specced

Resurfacing was core feature #7 in my own brief: items untouched for 14 days appear in a "revisit" widget. Six of the seven features shipped. I cut the seventh.

Not because it was hard. It was the cheapest thing left on the list, two rules over a timestamp. I cut it because **surfacing items on an age rule guesses at intent the product has no evidence for**. "You saved this a fortnight ago" isn't a reason to read something. Retrieval already answers "find the right thing fast", and a widget nagging about old saves answers a question nobody asked. It goes back on the roadmap the moment usage shows people hunting for things they'd forgotten they had, and I've written down what that evidence would look like.

DECISION 05

A hard \$0 budget, treated as a design constraint

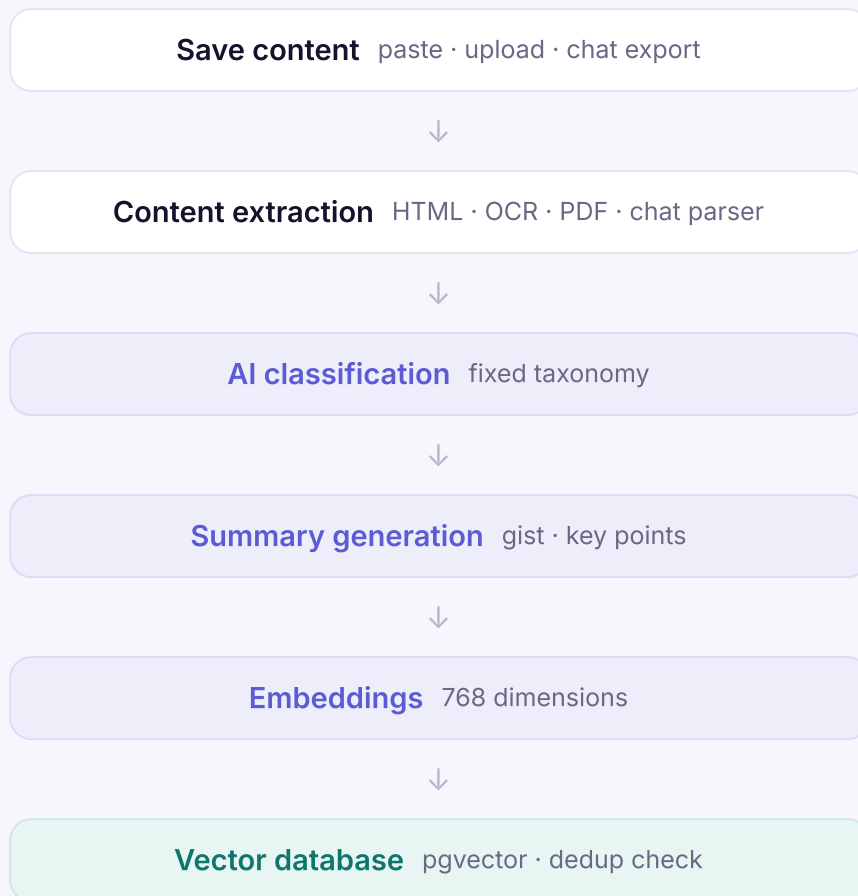
Every piece runs on a free tier: Supabase for Postgres, pgvector, auth and storage; Vercel for hosting and serverless functions; Google Gemini for generation and embeddings; OCR and PDF rendering in the browser, where they cost nothing.

That constraint shaped the architecture, not just the bill. Uploads cap at 4MB because a serverless function rejects a larger body before my code ever runs, so a higher limit would only swap a clear message for an opaque failure. A daily cron pings the database because free Supabase projects pause when they go idle, and a portfolio link is most likely to be opened by exactly the person you least want waiting on a cold start.

AI architecture

There are two AI paths and they are deliberately kept apart. One runs when content arrives, the other when a question gets asked. They share an embedding model and nothing else.

INGESTION: WHEN CONTENT ARRIVES



RETRIEVAL: WHEN A QUESTION IS ASKED



What each stage does

- **Extraction is per type.** Sanitised HTML for a note. Tesseract.js OCR in the browser for a screenshot. `pdf-parse` for a document, plus pdf.js rasterising page 1 as cover art. A parser that splits a chat export into one item per message, and an HTML fetch with metadata parsing for a link.
- **Classification and summarisation happen in one call.** Gemini returns strict JSON: category from the fixed list, a short title and subtitle, a 1 to 2 line summary, and 3 to 5 key points. The response is defended rather than trusted, so malformed JSON gets recovered, arrays get coerced, and an empty response raises instead of quietly saving a blank.

- **Duplicate detection reuses that embedding.** Cosine similarity against existing items, surfaced *before* saving, while it can still change your mind.
- **Only referenced sources become citations.** Retrieval returns the top matches; the answer cites some of them. Attaching all would inflate the "used N sources" claim into something untrue.
- **Citations keep their index and their passage**, so a reopened conversation still agrees with its own text and the exact passage an answer leaned on stays checkable.

Designing for the model failing

The model is treated as an unreliable dependency rather than a given, because in practice it is.

FAILURE	RESPONSE
Model overloaded (503)	Retry with backoff, then fall back across models. Another lightweight model first, full Flash last, because its free-tier allowance is roughly 20 requests a day
Daily quota exhausted (429)	A distinct error, because the advice differs: wait a minute versus wait until tomorrow
Malformed JSON	Recovered field by field rather than discarding the whole response
Classification fails entirely	The save still goes through and the item lands with editable fields. The AI accelerates this flow; it doesn't gate it
Embedding fails	Retried. An item without a vector is silently invisible to search and skips dedup, which is worse than a visibly missing summary

THE PRINCIPLE UNDERNEATH

Every one of these degrades toward the same outcome: *the user still gets their content saved*. The one thing that must never degrade is the citation. An answer you can't check is worse than no answer at all, because it looks exactly like one you can.

What broke, and what I changed

Four failures worth writing down. Each one changed how I think about shipping AI features.

– The app blamed the user for Google's outage

Adding content started failing with *"The AI couldn't read this one."* The content was perfectly fine. Google's `gemini-flash-lite` was returning `503, high demand` on five consecutive calls.

Three separate mistakes compounded here. The code used one model with no fallback. It gave up on the first failure with no retry. And it logged nothing, so a sustained upstream outage looked identical to unreadable content with no trace anywhere to tell them apart. Worst of all, the message blamed the user's input for someone else's traffic spike.

Now: retry with backoff, a fallback chain across models, every failure logged, and a distinct error for "the service is busy" versus "you're out of quota", because the advice differs. The fallback order is deliberate. Another lightweight model comes before full Flash, which allows only about 20 requests a day on the free tier, so leading with it would trade a short outage for an exhausted daily quota.

– A citation that pointed at the wrong number

An answer's inline markers count an item's position in the *retrieved* set, so a turn citing only the fourth match legitimately reads [4]. Reopening a saved conversation renumbered the sources from 1, which meant the text said [4] next to a source card labelled 1.

Worse, the restored view showed the item's *summary* where the passage the answer had actually used should have been. For a product whose whole promise is checkable answers, a citation you can't verify is the failure that matters most.

Now: citations are stored with their index and the retrieved passage rather than just an item id. The item itself is still looked up live, so edits show their current state and deletions drop out cleanly.

– A bug that only production would have found

Between analysing and saving, extracted text sits in memory. If that cache misses, commit re-extracts, which is sensible enough. But it kept only the text and threw away the metadata alongside it, so a note could save **with no body at all**, a link with no preview image, a PDF with no page count.

Locally this only fired if you left the review step open for fifteen minutes. On serverless, analyse and commit aren't guaranteed to land on the same instance, so the rare path becomes the common one. I caught it while reading the code for deployment, before it ever ran.

– Every uploaded file was publicly readable

The storage bucket was public. Any PDF or screenshot could be read by anyone holding the URL, no session required. That was harmless while the thing ran on my laptop. It stopped being harmless the moment it had a public address.

Now: the bucket is private and every file link is signed with a two-hour expiry. The path parser reads both the old public URLs and the new object paths, so flipping it needed no data migration.

What I'd take to the next build

Six things I believe more strongly now than when I started. Each one was paid for by something in this project going wrong.

1. An AI feature's error message is part of the feature

When Gemini went down, the app said *"The AI couldn't read this one."* The content was perfectly readable; Google was overloaded. That sentence sends a user off editing input that was never the problem. Failure copy has to name the layer that actually failed. And if the system can't tell you which layer that was, you have a logging gap rather than a wording one.

2. A citation is a promise, and the bar is verification

Citations that looked right were subtly wrong twice: renumbered on reopen so the text disagreed with its own sources, and showing an item's summary in place of the passage the answer used. Neither would fail a test or look broken. For a product whose whole claim is grounded answers, the standard isn't "looks cited". It's "can be checked".

3. Constraints did more for scope than any prioritisation framework

A hard \$0 budget and LinkedIn's terms of service each removed whole branches of the tree before I could argue with myself about them. Writing them down as locked decisions before building meant they held at 1am when the manual paste flow felt tedious. **The constraints I wrote down survived; the preferences I merely held did not.**

4. Cutting a specced feature is itself a decision

Resurfacing was in my own brief and was the cheapest thing left to build. I cut it because surfacing items on an age rule guesses at intent I had no evidence for. The uncomfortable part wasn't the analysis. It was that I'd already written the thing down, and written-down plans acquire a gravity of their own. Being able to say *why* it was cut, along with the evidence that would bring it back, is worth more than having shipped it.

5. Working locally proves less than it feels like it proves

One bug would have corrupted saves in production while being nearly invisible in development: between analysing and saving, a cache miss dropped the metadata and could store a note with no body. Locally it needed a fifteen-minute pause to trigger. On serverless, where consecutive requests hit different instances, it becomes the normal path. **Deployment is a different environment, not just a different address.**

6. Security posture belongs before the URL, not after

Every uploaded file sat in a public bucket. Fine on a laptop. Not fine the moment the app had a public address. Making it private and signing every link took under an hour, and it was the thing I'd have been most embarrassed to be asked about in an interview. **The moment something becomes reachable is the moment its weakest assumption becomes a real one.**

THE ONE THAT GENERALISES

Most of these are the same lesson wearing different clothes. **An AI product's hard parts sit around the model rather than in it.** The prompt was a day's work. Retries, fallbacks, honest failure states, verifiable citations, and knowing what to do when the model returns nonsense took the rest of the week. That part is what decides whether anyone trusts the output.

What I'd measure, and what I'd ask

DEFINED BEFORE THERE WAS ANYTHING TO MEASURE

These were written ahead of launch, so what counts as success isn't retrofitted to whatever the numbers happen to say later. Each metric names **the failure it would expose** rather than just the thing it counts, which is what makes it possible to be wrong.

The four numbers I care about

QUESTION	MEASURE	WHY THIS ONE
Does saved content get used?	% of saves opened at least once within 30 days	The core claim. If this doesn't beat the status quo of roughly zero, nothing else matters.
Does retrieval actually work?	% of vault queries where a cited source gets opened	An answer nobody clicks into is a plausible-sounding answer, which is the failure mode I'm most afraid of.
Is it faster than the old way?	Time from question to finding the right save	The product only earns its place if it beats scrolling LinkedIn saves.
Is the AI trusted or overridden?	% of AI-suggested fields edited at review	A high edit rate means the classifier is wrong. A zero edit rate probably means nobody's reading it.

The five questions I'd put to users

1. Show me the last five things you saved for prep. Where are they? *(Tests whether "scattered" is real or just my problem.)*

- 2. When did you last go looking for something you'd saved? Walk me through what you did.
- 3. What did you do when you couldn't find it? *(Re-searching Google instead is the strongest signal that saved effort is being wasted.)*
- 4. Here's an answer citing three of your saves. Do you trust it? What would make you check?
- 5. You haven't opened this in three weeks. Should the app have told you? *(The question that decides whether resurfacing comes back.)*

WHAT WOULD CHANGE MY MIND

If three of five describe hunting for something they knew they'd saved and then giving up, **resurfacing goes back in**. And it should be triggered by what they were looking for rather than by how old the item is.

If nobody clicks a citation, the trust model is wrong and the answer format needs rebuilding around the source rather than the prose.

How it's built

Frontend	React (Vite) + Tailwind, deployed on Vercel
API	Express, running as a Vercel serverless function on the same domain, so the frontend and API are same-origin and no CORS exists in production
Database	Supabase Postgres with <code>pgvector</code> for similarity search
Auth	Supabase Auth, Google plus email/password. Every table carries <code>user_id</code> and row-level security
AI	Gemini for classification, summarisation and answers; <code>gemini-embedding-001</code> at 768 dimensions for retrieval
In the browser	Tesseract.js for screenshot OCR, pdf.js to rasterise a PDF's first page. Both free, and both off the server

Two choices worth calling out. **The API is a serverless function on the same domain as the frontend**, so production has no cross-origin requests at all and CORS exists only for local development. And **OCR and PDF rendering run in the browser**, which keeps them off the server's clock and off the bill entirely.

Product Requirements: MVP

This is the brief the build was actually run against, written before the first commit rather than reconstructed afterwards. It's the reason the scope decisions above have dates and rationale attached. They were argued once, written down, and then honoured.

1. Problem & opportunity

Self-directed learning material is saved across LinkedIn, group chats and bookmarks with good intent, then rarely revisited. Even on a revisit it's hard to judge relevance without re-reading, or to find the right item at all. Saved effort quietly gets wasted. The MVP addresses this for Product Management learners, whose content types and vocabulary are predictable enough to classify well. That is the initial learning domain rather than the intended limit.

This has an architectural consequence: summaries are stored *alongside* full content rather than generated on demand, and "find the right thing fast" is prioritised over "store everything", since a plain folder app already does the latter.

2. Target user

Long-term: anyone who saves content to learn from later. Self-directed learners, career switchers, people ramping into a new role or domain.

Initial audience (MVP): Product Management learners. A large, well-defined first user segment with predictable content types and a shared vocabulary, which is what makes a fixed taxonomy and an accurate classifier possible at all.

Deliberately not yet: other roles and domains. Serving them is a taxonomy and onboarding problem rather than an architectural one. See the roadmap.

3. Goals and non-goals

GOALS	NON-GOALS
One place that accepts every format prep content arrives in	Being a general-purpose read-later app
Judge an item's relevance without re-reading it	Replacing the source platforms
Answer questions across the whole library, with citations	Answering from general knowledge rather than the user's saves
Catch near-duplicates before they accumulate	Collaboration, sharing, multi-user features

4. Scope decisions (locked before building)

- **No live LinkedIn scraping.** Manual paste of post text or URL only, because scraping saved posts violates LinkedIn's ToS.
- **No live WhatsApp listening.** The user exports a chat with WhatsApp's own feature and uploads the `.txt`, which is parsed into individual items.
- **Single user for the MVP.** No sharing, but every table carries `user_id` from day one so multi-user isn't a rewrite later.
- **Resurfacing cut entirely** (2026-08-06, after the other six features shipped). See Decision 04.

5. Feature scope

#	FEATURE	STATUS
1	Manual ingestion: text, link, image (OCR), PDF, WhatsApp export, interview question, job posting	Shipped
2	Categorisation & summarisation on ingest	Shipped
3	Embedding generation into pgvector	Shipped
4	Library: list, filter, search, sort	Shipped
5	RAG chatbot with citations back to the source item	Shipped
6	Duplicate detection above a similarity threshold	Shipped
7	Resurfacing widget	Cut , see Decision 04

6. Data model

TABLE	PURPOSE
<code>items</code>	Source type, raw and extracted content, summary, key points, category, timestamps, and type-specific columns (page count, word count, job fields)
<code>embeddings</code>	Chunk text plus its 768-dimension vector, per item
<code>item_categories</code> , <code>tags</code>	Multi-category filing; tags also carry favourites
<code>duplicates</code>	Near-duplicate pairs with the cosine score that flagged them

TABLE	PURPOSE
annotations	Highlights and notes, anchored by character offset with quote and surrounding context so they survive edits
chat_sessions, chat_queries	Conversations, structured answers, and citations with their index and retrieved passage

7. Taxonomy

Interview Questions (Product Sense · RCA · Metrics · Strategy · Behavioural) · Job Postings · Application Tips · Frameworks · Industry News · Other.

8. Explicitly deferred

- LinkedIn or WhatsApp live sync
- Multi-user accounts and sharing
- Resurfacing of any kind, rule-based or predictive

9. What this project demonstrates

- ▶ **Problem definition:** diagnosing why saved content goes unused, and rejecting the obvious "better folders" answer
- ▶ **MVP scoping and tradeoffs:** a deliberate first user segment, four locked constraints, one spec'ed feature cut with its reasoning
- ▶ **End-to-end AI product design:** ingestion, review, retrieval and verification as one experience
- ▶ **RAG retrieval workflow:** extraction, embeddings, vector search and grounded generation, built rather than described
- ▶ **Designing for trust:** citations that resolve to the exact passage used, and answers that degrade honestly when the model fails
- ▶ **Feasibility against user value:** a \$0 architecture, browser-side OCR, and limits chosen to fail with a clear message
- ▶ **Independent delivery:** idea to deployed product, including auth, security posture and a live demo

10. Roadmap

Not built, and each one listed with the reason it isn't in the MVP rather than as a wish.

Next: the friction the MVP still leaves in

WHAT	WHY IT MATTERS
Share directly from any app	Every save is currently copy, switch app, paste. The mobile share sheet is where saving actually happens, and removing that step matters more than anything else here, because the whole premise is that saving stays effortless.
First-run onboarding	A new account opens onto an empty vault. Nothing to search, nothing to ask. Onboarding should land one real item and one answered question before asking anyone for trust.
Export the vault	What's saved here can only be read here. Export makes the tool safe to adopt, and it belongs <i>inside</i> the reset and delete flows. A download offered before destroying a library is what makes that action reversible.
Browser extension	The desktop half of the same problem the share sheet solves.

Then: beyond the first domain

WHAT	WHY IT MATTERS
Any role, any domain	Nothing in ingestion, embedding, retrieval or citation is domain-specific. Only the taxonomy is. Making categories configurable, or inferring them from what someone saves, turns the MVP into a general learning system. It needs onboarding that asks what you're learning, and a classifier that works without a hand-written category list.
Practice mode	The saves already hold questions, frameworks and worked answers. Turning them into drills and spaced repetition is what separates a library from a study tool, and it's the most natural reason to come back.
Collections	Grouping saves into a plan ("week 3: metrics") gives the vault a shape that a flat list and a category filter can't.

Later: needs other people in it

WHAT	WHY IT MATTERS
Shared vaults	Study groups already share this material in group chats. A shared vault is where the export feature and multi-user meet.

WHAT

WHY IT MATTERS

Anonymised "what's trending"

The network effect available here: the same embeddings that power personal search would cluster across accounts to show what a group is studying. Needs multi-user, a privacy model, and enough users for an aggregate to mean anything.

SEQUENCING

Next reduces friction on a product that already works; **Then** widens who it works for; **Later** needs users to connect, so it can't be pulled forward however appealing. The one thing I would not build without evidence is resurfacing. See Decision 04.

PM Content Vault: built and shipped as a solo project, 2026.

Live app (<https://pm-content-vault.vercel.app>) · Source (<https://github.com/RADHIKA2909/pm-content-vault>) · Demo login `demo@pmvault.app` / `PMVaultDemo2026!`